0. Triplet overview and integration

Written by the Twin, City Brain and Guardian teams of Triplet 03 · shared data dictionary `2026-09-17` and the Guardian's contract 1.0 · `INTEGRATION.md`

0.1 Architecture and how the layers talk

Three teams build one city. The **Twin** simulates it and is the only layer that knows what really happens. The **Guardian** judges every reading and audits every actuator call. The **City Brain** detects incidents and decides, acting only on readings the Guardian has trusted. Each layer runs alone with mocks of the other two (R1), starts every scenario from one trigger and logs it (R2), and shows its events live (R3).

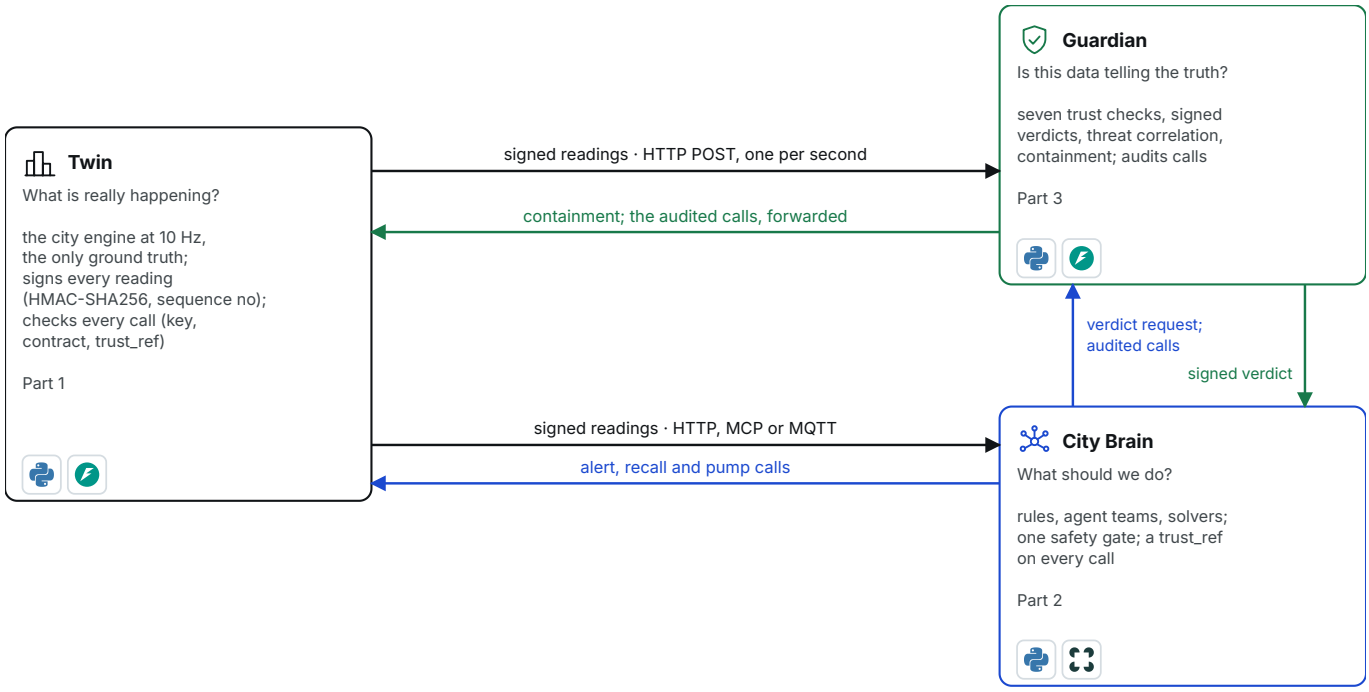


Figure 0.1. The triplet as `INTEGRATION.md` and the three Interface Cards define it. Grey: readings; blue: the Brain's calls; green: the Guardian's answers and containment. Every address is configuration (`TWIN_URL`, `BRAIN_URL`, `GUARDIAN_URL`).

Link	Message	Transport	Protection
Twin → Guardian	every signed reading	HTTP POST, one reading per request, every second	HMAC-SHA256 over canonical JSON; a rising sequence number per sensor
Twin → Brain	every signed reading; the city layout	the Brain pulls them: MCP `list_readings`, or HTTP	the same signature and sequence checks, in the Brain
Brain → Guardian	a verdict request by sensor and sequence number	REST `POST /v1/verdicts/request`, 2 s deadline	an API key per layer; the verdict is HMAC-signed
Brain → Twin	a signed actuator call with its `trust_ref`	dispatch through the Guardian's audit; alert, recall and pump straight to the Twin's MCP tools	the Brain's gate, the Guardian's auditor, the Twin's key and `trust_ref` checks
Guardian → Twin	isolate, quarantine, clear	a call with `requesting_layer: guardian`	containment is Guardian-only on the Twin; caps and release paths

0.2 One scenario end to end: a robbery and a pursuit

A robbery is caught on camera (scenario T14). The trace is the committed run `logs/t14-20261108T153858Z.jsonl` at normal speed, with times measured from the robbery report. The Twin and the Guardian in this run are the Brain's stand-ins, which speak the shared contract; the three teams' real layers have also run together (0.3).

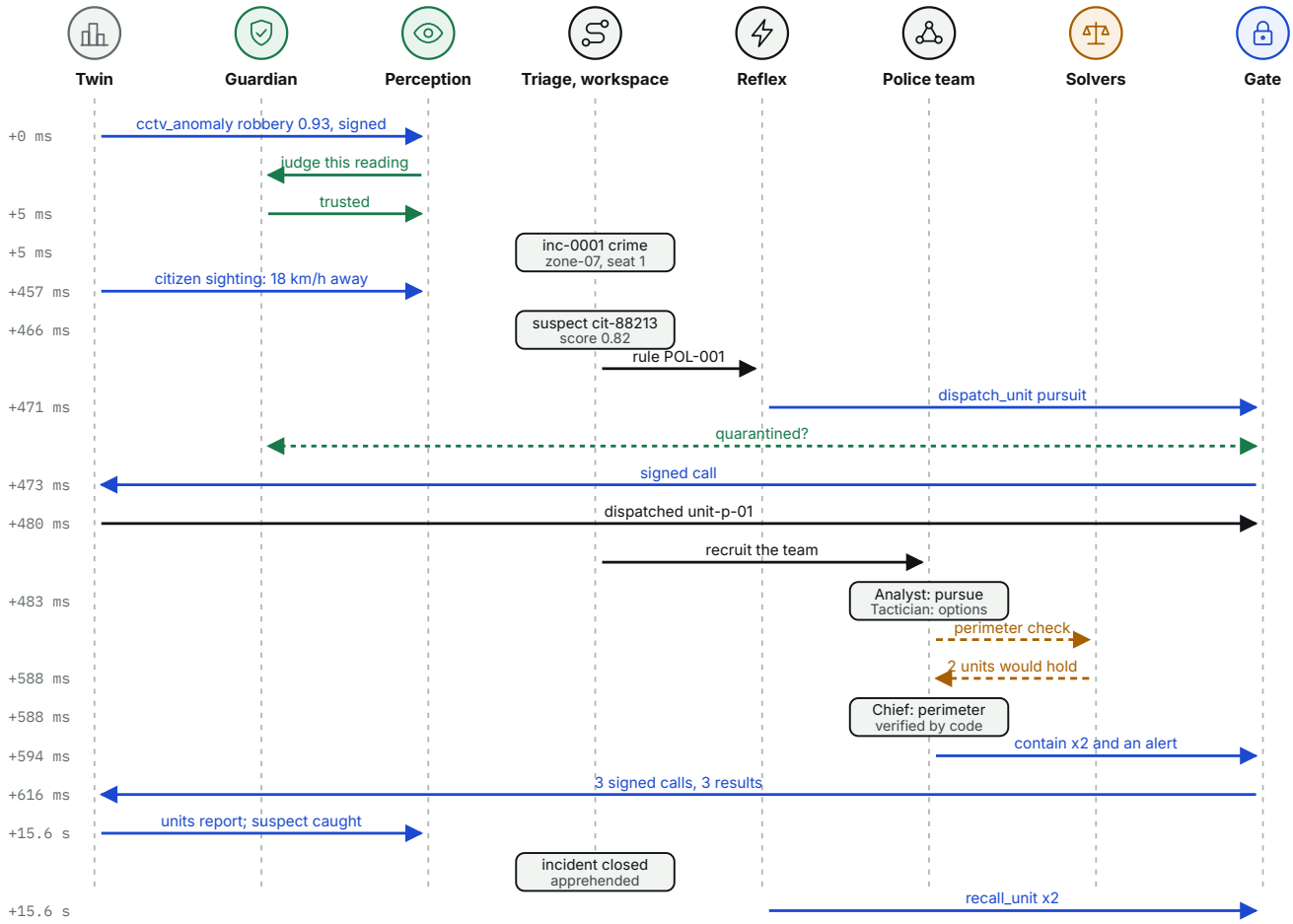


Figure 0.2. T14 across the layers. Every reading is judged before it counts; the reflex dispatches one unit 15 ms after the reading that completes the evidence, and the team's solver-checked perimeter follows about 120 ms later through the same gate.

Layer	What it does in this scenario
Twin	streams the signed camera reading and a citizen running away at 18 km/h; answers each dispatch with a unit id; reports the units on their way and the arrest
Guardian	judges every reading within about 5 ms; a call to a station it had quarantined would be refused (T6, T16)
City Brain	opens the incident, scores the suspect 0.82, dispatches one unit by reflex, adds two perimeter units and an alert, and recalls every unit at the arrest

Each part also traces a scenario of its own team: an accident from the truth to its log (Figure 1.5), one reading through the Brain's eighteen steps (Figure 2.2), and a fake flood at the port (3.2.1).

Where to find each of the jury's criteria

Criterion (points)	Sections
Individual prototype functionality (10)	1.2, 2.2, 3.2; the scenarios of Part 4
Technical implementation and features (5)	1.3, 2.3, 3.3
Interoperability between the three teams (15)	0.1, 0.3; 1.4, 2.4, 3.4; each layer's `INTERFACE.md`
Security and trust mechanisms (10)	1.5, 2.5, 3.5; the gate (Table 2.1) and the trust checks (3.2.1)
Documentation and code quality (5)	1.6, 2.6, 3.6; each layer's `README.md`
Demonstration and presence (5)	0.2; the triggers and logs of Part 4; the three timeline screens

0.3 Shared decisions

- **One data dictionary**, contract `2026-09-17`, with an example of every message in each layer's Interface Card.
- **Signed readings.** The Twin signs each reading with HMAC-SHA256 over canonical JSON (sorted keys, no spaces) and a sequence number per sensor; the Guardian and the Brain both check it.
- **No verdict, no action.** Every Brain call carries a `trust_ref` to a trusted, unexpired Guardian verdict, checked by the Brain's gate, the Guardian's auditor and the Twin.
- **Containment belongs to the Guardian**, and valve, grid, signal and dispatch calls pass its audit before they reach the Twin.
- **Configuration, not code.** Addresses and keys come from configuration and the environment.
- **One numbering, one log.** T1 to T20 across the three teams, one trigger and one log per run (Part 4).

When a layer fails

Failure	What happens	Shown by
The Guardian stops answering	the Brain acts on nothing the Guardian has not judged; the Twin keeps running	T15
The Twin goes silent	the Guardian marks every sensor unknown; the Brain reports it and keeps its state	T7; Brain tests
A partner is unreachable for the Twin	the delivery fails and is logged; the city keeps running	Part 1, 1.4
A forged or replayed reading	the Guardian marks it untrusted; the Brain refuses it with a code	T3; Brain red team
A device is quarantined	the Guardian refuses calls to it; the Brain picks another station	T6, T16

Integration status

On 8 November the three teams' real layers ran together as local processes, with no mock and no person deciding. A pedestrian hit, a car crash and a building fire went end to end, each unit dispatched through the Guardian's audit and each incident closed; a flood opened once, with a public alert (`twin/docs/joint-run-2026-11-08.md`). Every link now follows the shared contract:

Link	Agreed and built
Readings to the Brain	the Brain pulls them from the Twin: MCP `list_readings`, or `GET /readings`
Readings to the Guardian	the Twin pushes each signed reading to `POST /v1/readings`
Verdicts	the Brain asks `POST /v1/verdicts/request`; the Guardian answers with a signed verdict
Actions	dispatches pass the Guardian's audit and reach the Twin at `POST /tools/{action}`; alerts, recalls and the pump go straight to the Twin's MCP tools
Still to do	the same runs through the root `docker-compose.yml`, with the Brain's agent teams (the run used its rules), and the pump call, added after the run, played with all three

1. The City (Twin)

Written by the Twin team · folder `twin/` · contract `2026-09-17` (the Guardian team's dictionary) · "What is really happening?"

1.1 Role and goals

The Twin **is the city**. It simulates two city blocks in real time: cars, pedestrians, signals, three emergency units, 24 buildings and the rain on seven streets. It is the only layer that knows what really happens. The City Brain and the Guardian see it as a real control room would: through sensor readings the Twin signs, and through the actuators they call. The Twin never dispatches by itself; an ambulance leaves its bay only when a `dispatch_unit` call arrives.

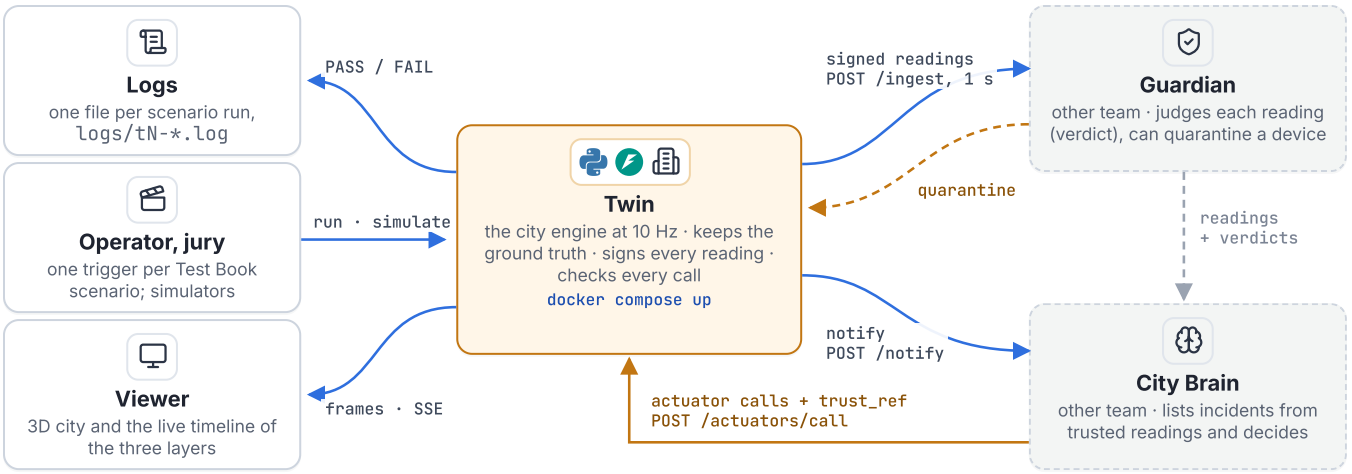


Figure 1.1. The Twin and its partners, with the protocol on each link (HTTP and JSON). Blue: what the Twin sends; amber: what other layers ask it to do. Dashed cards are the other teams' layers: mocks here, until `GUARDIAN_URL` or `BRAIN_URL` points at a real one.

Four rules shape the design:

1. **The truth stays inside.** The engine records every accident, fire and flood, but the other layers learn of it only from sensors, which can be late or wrong: a caller places a crash 3 m off, and one in ten names the wrong category.
2. **Nothing moves without a call.** Units, road barriers and pumps change only through actuators, and every call is checked: the layer's key, the contract version, the parameters, and that a Brain call names the Guardian verdict it relies on (`trust_ref`).
3. **Every reading can be checked.** Each one carries a sequence number per sensor and an HMAC-SHA256 over its canonical JSON, and honest sensors agree with each other (rain against water level, smoke against air quality).
4. **One trigger, one log.** A scenario starts with one button or command, is checked against the events expected from all three layers, and ends with `PASS` or `FAIL` in its log (R2).

Today the Twin runs alone with its two mocks from one command, and the four Test Book scenarios pass end to end through them. It has 303 Python and 94 Vitest tests; the traffic rules are checked from geometry, not engine internals (`tools/check_rules.py`).

1.2 Architecture

The Twin is one Python service around a seeded engine. Every 0.1 s the engine advances the whole city by one tick; the service turns what happened into readings and live frames, and incoming calls into changes in the engine. The API talks only to a `CitySource` interface, so another city or another transport (MCP) is a new adapter, not a rewrite.

From the Phase 0 design to what runs

Our Phase 0 report planned a Twin on SUMO, MCP servers and mutual TLS. We kept its ideas (subjects with traits, intent apart from physics, sequence numbers and HMACs, an LLM teacher distilled into a student) and changed the means, so its diagrams no longer describe the code and are redrawn.

Phase 0 plan	What runs now	Why we changed it
SUMO through TraCI for driving	our own engine: IDM, MOBIL, a kinematic bicycle model	junction, crosswalk and siren rules had to be ours; no second process to keep in step
External, Control and Logs MCP servers	HTTP and JSON, server-sent events for streams, behind a seam	our decision of 14 Oct, to keep the first link simple; MCP stays one adapter away (§1.4)
NDJSON log stream over mutual TLS	one signed reading per request, HMAC and sequence number per sensor	the Guardian's contract asks for exactly this envelope; TLS is still open (§1.6)
LLM teacher, XGBoost student, LLM fallback and canary checks	the teacher and the student; a random choice if the student cannot score	no network or API key on stage; a demo cannot depend on an LLM answering

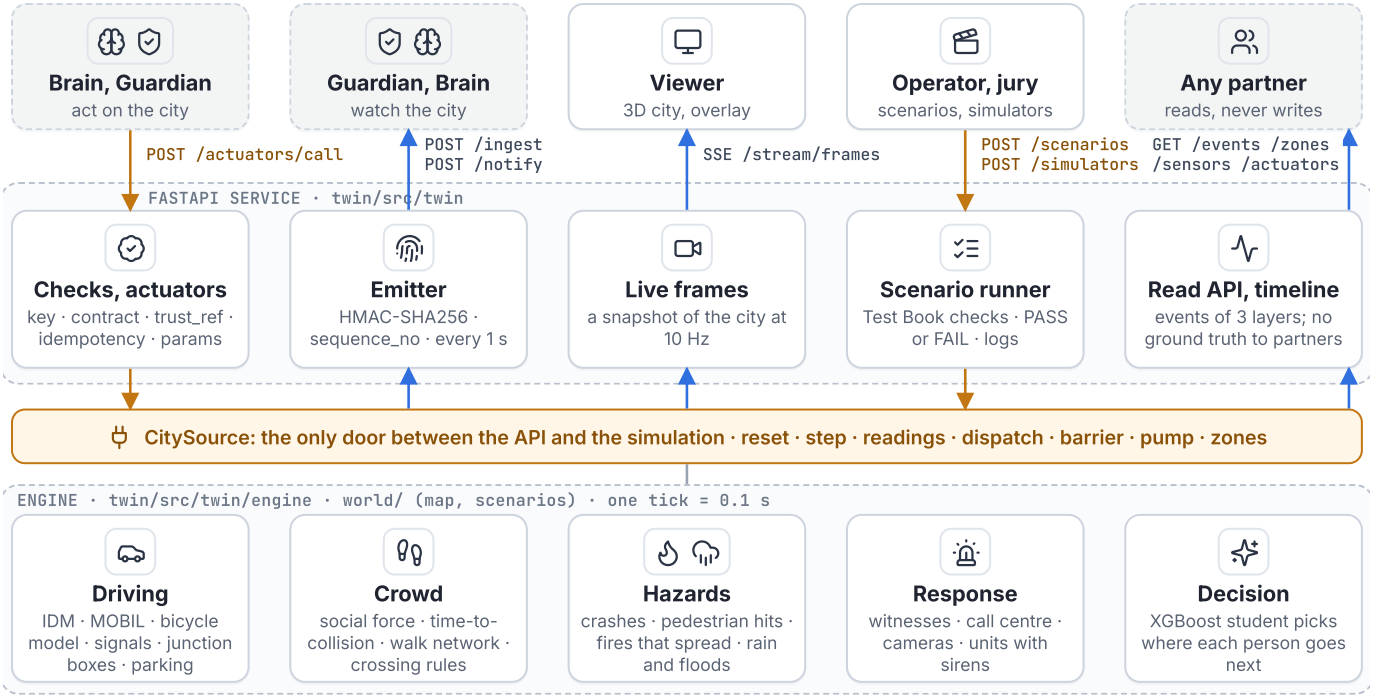


Figure 1.2. Inside the Twin. Five columns, five paths: acting (amber, down), sensing (blue, up), showing, testing, reading. Every path crosses the `CitySource` seam (amber), so the engine never sees HTTP and the API never sees engine objects.

1.2.1 City plan and zones

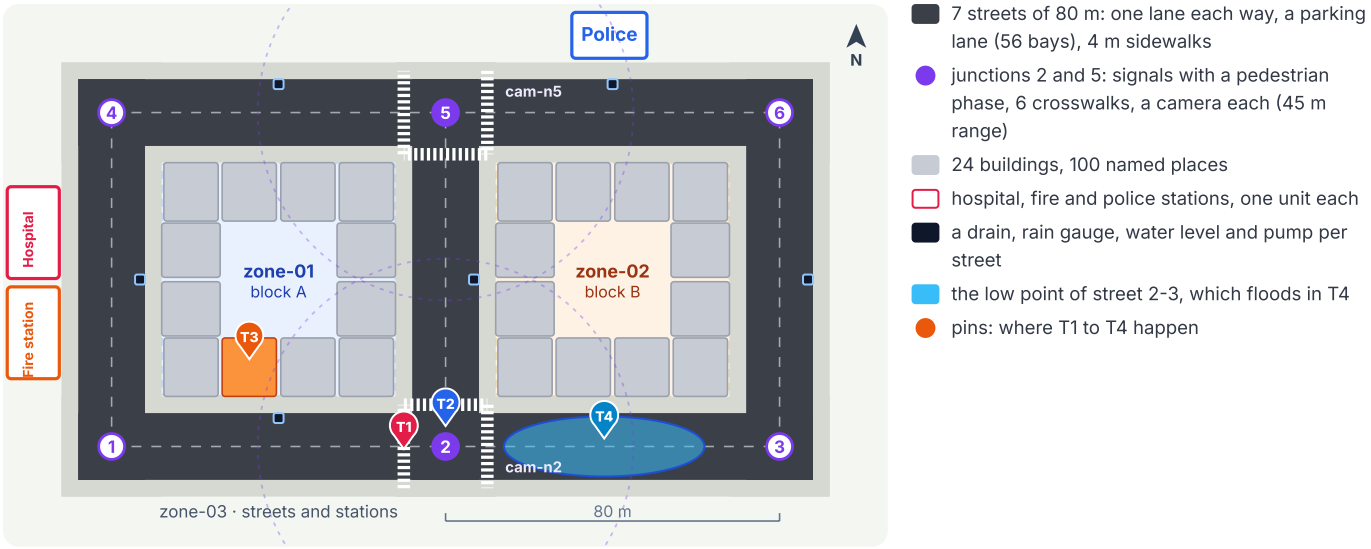


Figure 1.3. The two-block map, drawn to scale from the Twin's own map (`GET /live/map`). Three zones answer the contract's open item on granularity: `zone-01` and `zone-02` are the blocks, `zone-03` the streets and stations; `GET /zones` gives their bounds, adjacency and facilities, so no layer hard-codes geometry.

1.2.2 One tick of the city

The order inside a tick is fixed, and it decides who reacts to what. Signals and water are settled before anyone moves, accidents are found after everyone has moved, and observers look last.

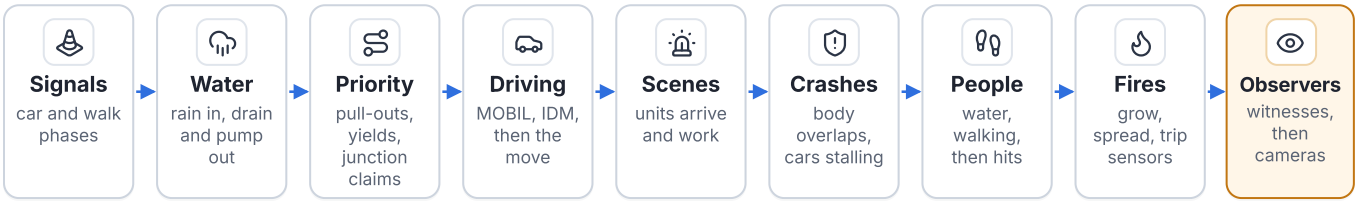


Figure 1.4. One `Simulation.tick`, 0.1 s of city, left to right. What the observers report is all the other layers will ever learn. Fires, floods and witnesses draw from their own random generators, so arming a hazard does not change the traffic of a seed.

1.2.3 Simulation model

Each engine is a small module with one job, wired together in `sim.py`. We chose published models where they exist, and the rules are checked by tools that measure the geometry of a run.

Part	Model	What it gives the city, with its main numbers
Cars	Intelligent Driver Model; MOBIL lane changes; kinematic bicycle steered by pure pursuit (Stanley when parking)	50 km/h, 1.5 s headway; cars slow for turns, stop at red and yellow lines, park in real bays and pull out only into a gap
Junctions	fixed-time signals, two car phases then a pedestrian phase (44 s); a junction box per conflicting movement	a car enters only when its exit is clear and nobody is in its path; no two bodies overlap in normal traffic
Pedestrians	social-force goal term; time-to-collision avoidance (Karamouzas et al. 2014); a walk network of sidewalks and crosswalks	1.4 m/s walkers, 3 m/s runners; they cross only on crosswalks, on `walk` and with a gap of 4 s or 10 m
Accidents	body overlap (separating axis) between cars, a car rectangle against a person's disc	reckless road users from two risks: red-light runners crash, non-yielding drivers and careless walkers cause pedestrian hits
Fires	a fire grows from 0 to 1 in 30 s; after 40 s it tries to spread to touching buildings every 15 s (50%)	smoke sensor trips at 3 s; 20 to 400 °C and 420 to 5000 ppm CO_2; drivers and walkers keep 25 m away when they can
Floods	inflow by the rational method Q = C·i·A; a parabolic dip, V = 4/3·W·L·$d^{1.5}$/√H; drain 0.3 m³/s, pump 1.2 m³/s	at 0.20 m a street is closed to cars; walkers turn back at 0.10 m on the sidewalk; at 0.20 m a person is trapped
Units	siren driving: ignore signals (7 m/s through a red), 19 m/s top, claim a junction 45 m out, pass queues on the centre line	out of the bay, to the scene, work it (treat, load, close the road, put out, rescue), hospital, back to the bay
Witnesses	line of sight within 40 m; a reaction delay of 1 to 8 s; a call centre that merges calls within 50 m and 300 s	the location 3 m off (10% of callers 15 to 30 m off), 10% name the wrong category; false reports can be switched on

1.2.4 How a scenario is played

A scenario is random traffic plus a short script that stages one event, fired by what the actors are doing (a light about to change, the water 0.27 m deep) rather than at a fixed time, so it still happens with another seed. After that nothing is scripted: witnesses, calls, verdicts and the drive are the code of the free city. Figure 1.5 follows T1; the tags are what each step leaves on the timeline.

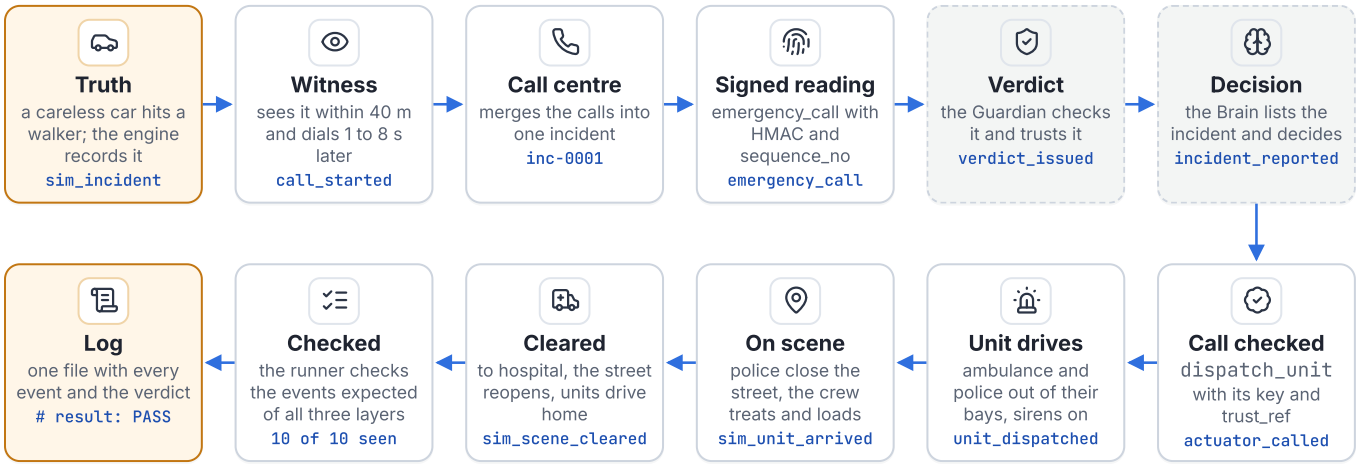


Figure 1.5. One scenario from the truth to its log (T1, about 245 s of simulated time, about a minute at 4×). Dashed cards are other layers. The Brain hears of the accident only when someone sees it and calls.

Every run is one button in the viewer or one command (`docker compose run --rm scenario t1`). The runner switches the mock Brain to decide by itself, or leaves it to a person at its console, waits for the expected events and writes the merged timeline of the three layers to `logs/`. Run for this report on commit `2dc94ec` with the mock Brain deciding, all four pass: T1 saw its 10 expected events and closed at 245 s, T2 10 of 10 at 287 s, T3 9 of 9 at 146 s, T4 12 of 12 at 340 s.



1. Live run or recording, its scenario and clock.
2. Scenarios, city settings, cameras and the People drawer.
3. Counts of cars, people and calls; a card per incident; the three units.
4. In the city: the accident, the call about it, each place by name.
5. Restart, pause, speed from 0.5× to 8×, and the time bar.

Figure 1.6. The viewer in a Busy city run at 8×. Cards say "Nobody is coming yet": no layer has dispatched a unit, so all three are still at their stations. It is the operator's window, drawn only from what the Twin streams.

1.2.5 Decision model: where people go next

As in Phase 0, the model only picks a destination and the engines do the rest. The student is a small model that copies what an expensive LLM (the teacher) would choose, so a decision makes no API call. We generated 6,000 synthetic

decisions: a person (7 traits, a role, driving or walking), the hour and place type, 12 candidate places (category, distance, open or closed, crowd, home or duty place) and 0 to 2 events (fire, crash, closed street). `gpt-4o-mini` answered each with one letter, and its token log-probabilities give a real probability over the 12 options, richer than a single pick (4 minutes, $0.30, no parse failures). Each (person, situation, candidate) triple becomes a row of 58 numbers, the facts in the teacher's prompt. XGBoost learned from 4,800 scenarios (58k rows) the probability that the teacher picks a candidate; 1,200 were kept for testing.

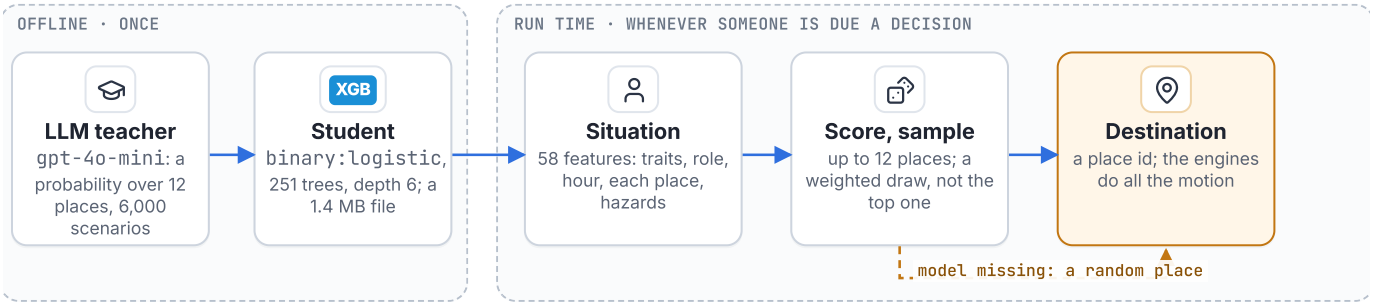


Figure 1.7. The sim scores the candidates and samples one in proportion, so people vary. A simulated day passes in 24 minutes from 08:00.

Match with the teacher, 1,200 held-out scenarios

	Top-1	Top-3
Student	**60.5%**	**90.5%**
Nearest open place	42.7%	not measured
Random	8.3%	25.0%

Share of probability on places to avoid

	Student	Teacher	Chance
Closed places	**10%**	6%	45%
Near an event	**9.1%**	8.1%	14.2%

`test_decision.py`: avoids closed places, fires, crashes, long trips.

1.3 Tech stack and why

At run time everything is on one laptop, with no cloud service. Each logo belongs to its owner.

	Job and technology	Why
	Language and contracts Python 3.12, uv, Pydantic	One language for the engine and the service; a locked build (`uv.lock`); the actuator envelope is a typed model that rejects a bad call with a clear message.
	Service FastAPI, Uvicorn, server-sent events	Typed endpoints with OpenAPI docs at `/docs`; JSON Schema for every actuator in `GET /actuators`; streams without a message broker.
	Simulation NetworkX, NumPy, OSMnx (optional)	Road, turn and walk graphs and their routes; vector geometry. The same engine loads Avenue Habib Bourguiba from OpenStreetMap.
	Decision model XGBoost (CPU build)	Scores every candidate place in one call; the plain wheel would add about 350 MB of GPU libraries, the CPU build does not.
	Viewer React 19, three.js, React Three Fiber, Vite	The 3D city and the timeline draw only what the Twin streams; the same code plays a recording made by the engine.
	Operations Docker Compose, nginx, pytest, Vitest	One command starts the Twin, the viewer and both mocks; nginx adds the operator key so it never reaches a browser; 303 Python and 94 Vitest tests.

1.4 Interfaces in/out

The Interface Card is `twin/INTERFACE.md`. Messages follow the Guardian team's draft dictionary (`2026-09-17`); fields we added are marked *proposed* there.

Sensors: what the Twin sends (rain and water per street)

Type	Sensors	When
`emergency_call`	`ecall-zone-03` …	on each call
`traffic_camera`	`cam-n2`, `cam-n5`	every second
`cctv_anomaly`	`cam-n2-cctv` …	on an anomaly
`building_fire`	`fire-bld-01-03` …	while it smokes
`air_quality`	`aq-zone-01` …	every second
`rainfall`	`rain-dip-2-3` …, 7	every second
`water_level`	`wl-dip-2-3` …, 7	every second

Actuators: what the Twin accepts

Action	From	Effect in the city
`dispatch_unit`	Brain	sends a unit to an `incident_id`
`set_barrier_state`	Brain	close or open a street; cars re-route
`set_pump_state`	Brain	a street's pump on or off (1.2 m³/s)
`quarantine_device`, `isolate_sensor`, `clear_quarantine`	Guardian	its readings stop until cleared; the device keeps working

What the jury looks for. Logic apart from partner I/O: the `CitySource` protocol in `city/base.py`, the only thing the API talks to. Settings in configuration: `config.py` reads the environment, and an empty address disables a link. Clear

errors: `400 401 403 409 422` with a sentence (`test_api.py`, `test_actuator_catalog.py`). Runs alone with mocks (R1): `docker compose up` (`test_interop_guardian_mock.py`, `test_pipeline_e2e.py`); with no partner at all it still simulates.

Direction	Transport and contract	Behaviour a partner can rely on
Out, to the Guardian	`POST {GUARDIAN_INGEST_URL}`, one signed reading per request, every 1 s; also `GET /stream/readings` (SSE)	sequence numbers rise per sensor; a partner that is down costs a `delivery_failed` event, never a stalled city
Out, to the Brain	`POST {BRAIN_NOTIFY_URL}` `readings_available` after each batch	no data in the notice: the Brain reads only what the Guardian has judged
In, from either	`POST /actuators/call` with `X-Api-Key`, `requesting_layer`, `justification`, `trust_ref`; optional `idempotency_key`	answers `dispatched`, `duplicate` or `rejected` with a reason; errors `400 401 403 409 422` with a sentence in `detail`

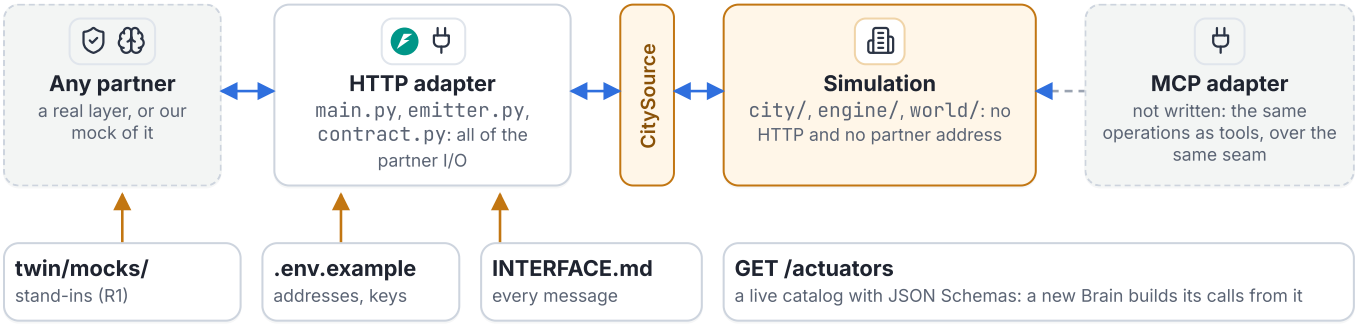


Figure 1.8. Where the logic ends and partners begin. Phase 2 reshuffles the triplets, so a new Guardian or Brain must need only a new address, or at most a new adapter, and never a change to the simulation.

1.5 Security measures

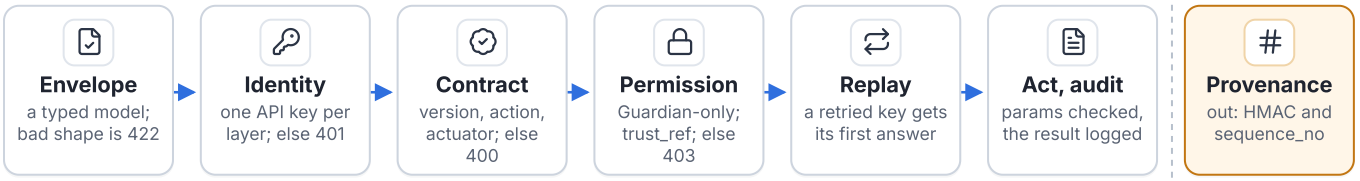


Figure 1.9. Six checks on every incoming call, in the order the code runs them; a call stops at the first that fails, with a sentence saying why. Right of the line: what protects outgoing readings.

Boundary	Control, and the test that keeps it true
Twin to Guardian: readings	HMAC-SHA256 over canonical JSON (sorted keys, no spaces, UTF-8) with `TWIN_HMAC_KEY`; sequence numbers strictly rising per sensor, so drops, replays and edits show (`test_signing.py`, `test_interop_guardian_mock.py`)
Brain and Guardian to Twin: calls	a key per layer, so `requesting_layer` is proven and not just claimed; a Brain call without `trust_ref` is refused (403) and containment is Guardian-only (`test_api.py`); refusals of a key or a permission are logged as `actuator_rejected`
Ground truth	what really happened goes only to the operator's `/timeline`; partners' `/events` leave it out, so a layer cannot cheat its way to a verdict
Operator, secrets	scenarios, simulators and live control need `X-Operator-Key`, which nginx adds so it never reaches a browser; all keys come from the environment, never from code

1.6 Known limitations

- **No joint run yet.** Every scenario passed against our mocks only. The Brain team plans MCP by default with HTTP as an alternative, and the Guardian's `/v1/readings`; addresses are configuration, but paths and event names still need aligning.
- **No falsified readings yet.** The truth is kept apart, but a fake reading cannot yet be sent on one trigger, as bad-data scenarios need; the Test Book has no layer-failure scenario either.
- **Trust stops at presence.** The Twin checks that `trust_ref` is there, not that the verdict exists; that needs a way to ask the Guardian.
- **No TLS.** Shared keys with no rotation and dev defaults in Compose; read endpoints are open, and so are the viewer's controls.
- **Part of the dictionary.** No power-load sensor, and no `set_signal_phase`, `set_valve_state` or `set_grid_connection`.
- **A small city.** Traffic rules are measured on two blocks only. Junctions trade about 23% of car trips for safety, and nobody drives round a wreck.
- **HTTP, not MCP.** The MCP adapter is not written. The student has no LLM fallback or drift check at run time, and its training pipeline is not in this repository.

2. The City Brain

Written by the City Brain team · repository folder `triplet-03/brain/` · contract `2026-09-17`, Guardian contract 1.0 · key question: "What should we do?"

2.1 Role and goals

The Brain reads the Twin's live sensor readings, detects incidents (crimes, fires, accidents, jams, floods, grid overloads), asks the Guardian whether each reading can be trusted, chooses a response and carries it out through the Twin's actuators. It acts only on readings the Guardian has marked trusted, and only through the safety gate, the one component allowed to call an actuator.

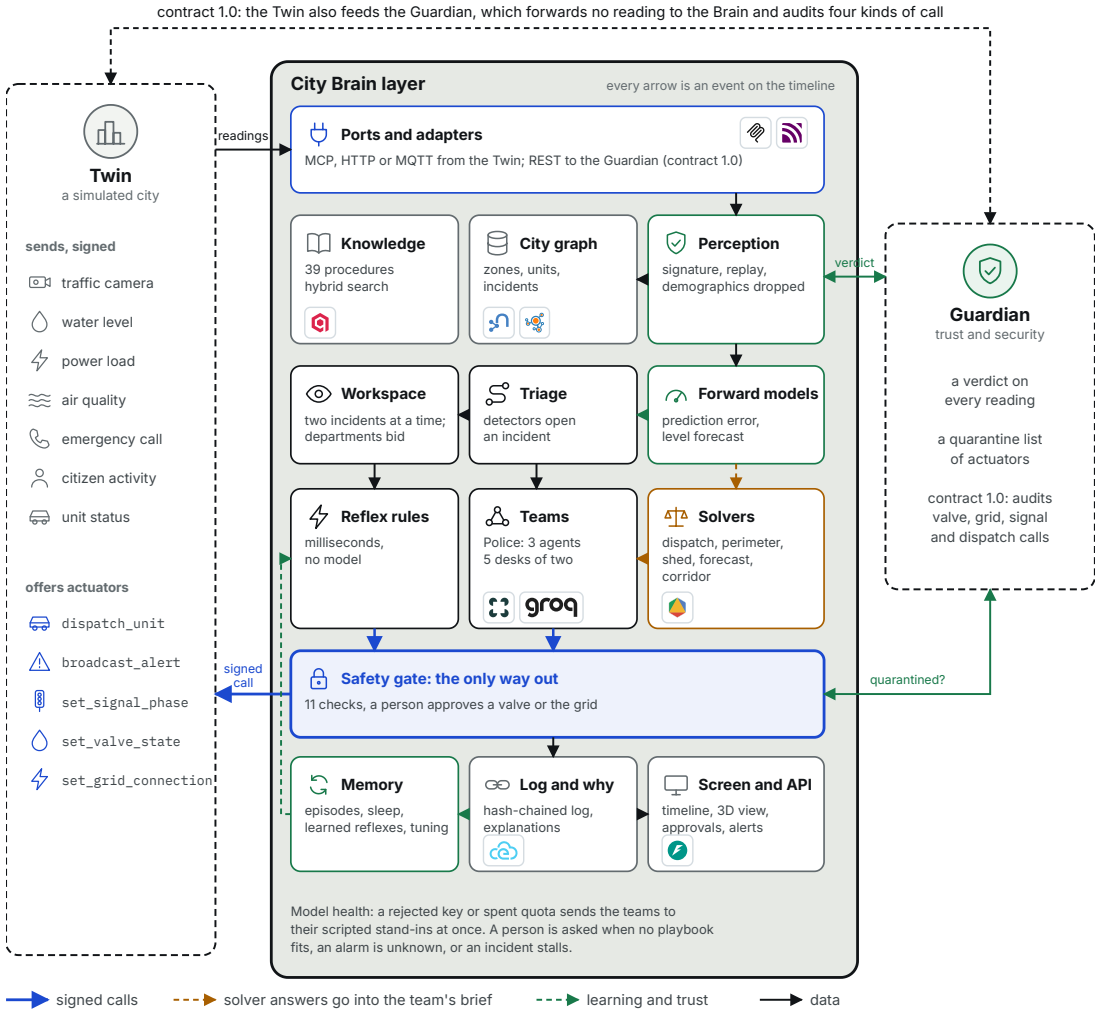


Figure 2.1. The Brain's components. Readings enter through the adapters at the top and actions leave only through the gate. With the Guardian team's contract 1.0 (dashed line) the Twin also feeds the Guardian, and valve, grid, signal and dispatch calls reach the Twin through the Guardian's audit. Amber dashes: a solver's answer enters a team's brief; green dashes: learning.

Goals, and how each is shown

Goal	How the Brain meets it	Shown by
Act only on trusted data	every call carries a `trust_ref` to a trusted, unexpired verdict	T13: a fake flood, nothing sent (Listing 2.1)
One way out	rules, agents and solvers propose; only the gate sends	`brain redteam`: 24 of 24 attacks refused
Fast first, then thorough	a reflex acts at once; the team's plan follows, checked by code	T14: reflex in 15 ms, team plan 123 ms later
Safe when a part fails	no verdict, no action; a scripted team replaces a lost model	T15: Guardian silent, nothing more sent
Every step explained	each decision, hold or refusal is a named event with its reason	chained log per run; the "why" API

All five Brain scenarios (T12 to T16) pass, with their logs in `logs/`; 1,114 tests; 60 of 60 held-out variants decide as their base scenario.

2.2 Architecture

Figure 2.2 follows one reading through the components of Figure 2.1, in eighteen steps from the Twin to an action and its explanation. Each card names the component, the technology that runs it and, in blue, the brain function that inspired it. The analogy guides the design; the Brain does not model a brain.

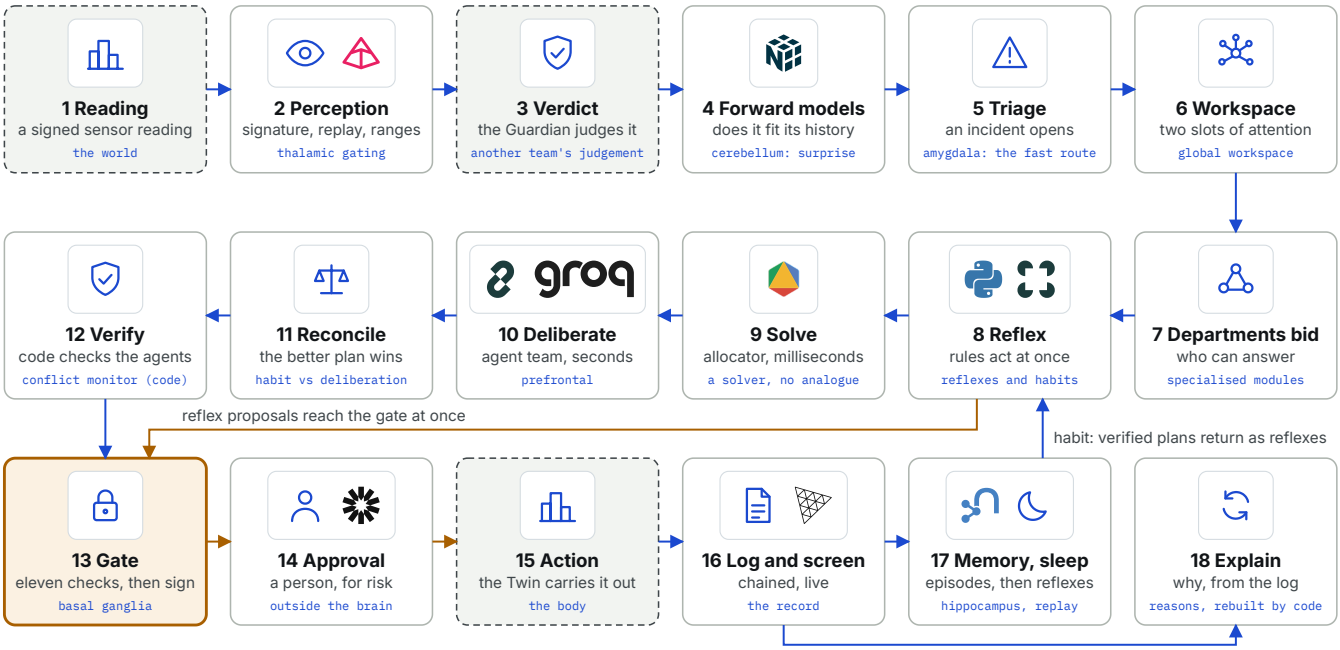


Figure 2.2. From a reading to a decision. Amber arrows lead to the gate: reflex proposals reach it at once, and the team's plan follows through the same gate. A plan the team makes the same way twice returns as a reflex after the sleep phase.

2.2.1 Incident detection

Perception checks the contract version, the HMAC signature, the sequence number and the value ranges, and refuses a bad reading with a code. **The Guardian's verdict**, requested with a 2 s deadline, decides whether the reading may count. **Forward models** learn each sensor's normal range from trusted readings only. A reading far outside it is flagged as a surprise, and the reflex it would trigger waits for a second source; a Kalman tracker forecasts when a river will reach the flood line. **Triage** runs the departments' detectors to open or join an incident, and the **workspace** gives two incidents at a time the team's attention, ranked by severity, trust, surprise and urgency.

When an alarm matches no department, an incident has no procedure, or an incident stalls, the Brain emits `brain.human.needed` and asks a person.

2.2.2 Decision method

Rules (14, in six packs) answer in milliseconds with no model. **Agent teams** weigh the case: for Police an Analyst, a Tactician and a Chief on LangGraph, using Groq models with `--llm` and a scripted stand-in otherwise or when the model fails; the other departments use a two-role desk. **Solvers** compute what should not be guessed: OR-Tools min-cost flow for dispatch, CP-SAT for a perimeter and for load shedding, Dijkstra for a green corridor and the Kalman forecast for floods. They run before the agents and their answers go into the agents' brief, so a model that never calls a tool still reads them.

Code then verifies the agents' plan (`verify_case`, `verify_plan`, `verify_desk`) before it reaches the gate.

Departments

Department	Incidents	Acts through	Reasoning
Police	crime	`dispatch_unit` (pursuit, contain), `broadcast_alert`, `recall_unit`	Analyst, Tactician, Chief
Fire, Ambulance	fire, accident	`dispatch_unit`, `broadcast_alert`	desk: Analyst, Chief
Traffic control	jam	`set_signal_phase` (hold green, also for an ambulance's route)	desk
Disaster response	flood	`broadcast_alert`, `set_pump_state`, `set_valve_state` (a person approves)	desk
Utilities	overload, smog	`broadcast_alert`, `set_grid_connection` (a person approves)	desk

A department is three files (detectors and actuators, reflex rules, a playbook): a new kind of incident is a new pack.

2.2.3 Choosing an actuator

The Brain picks an actuator in three steps. The incident type decides which department may act and with which actuator; a solver, or the nearest free station, picks the device; and the gate decides whether the call goes out. No other code path calls the Twin.

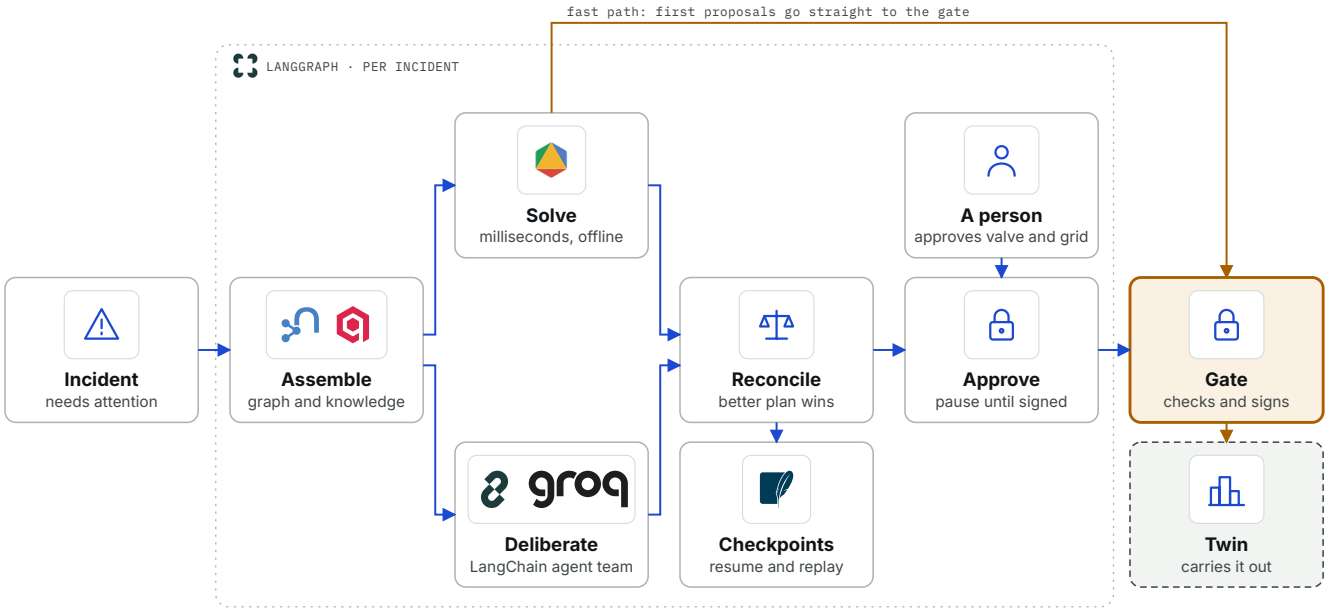


Figure 2.3. The planning graph for one incident (LangGraph). The solver and the agent team work in parallel, the better plan is kept, a valve or grid call pauses for a person, and each step is checkpointed to SQLite so a run can resume and be replayed.

Station or device

The nearest free station by hop distance, never a quarantined or reserved one. In T16 the Guardian quarantines a station, the gate refuses calls to it with reason `quarantine`, and another station is sent. When two incidents need the same kind of unit, the dispatch solver plans both together, and its choice is used only if it costs less: for two collisions it may send the far ambulance to the first so that the near one stays free for the second. On 100 random cities its plans cost 17% less than nearest-first (20.39 against 24.63) and match brute force (`brain optimise-bench`).

Grid cut or signal

A cascade analysis weighs what each grid cut would black out, with a hospital weighted 10, and the shed solver picks the cheapest set of cuts that removes the excess load. A corridor holds the signals on an ambulance's route at green. A dispatched unit's `unit_status` readings close the incident, and an arrest recalls every unit still out.

Table 2.1. The safety gate's eleven checks, in the order they run on every proposal. The first that fails stops the call and its reason goes to the timeline; quoted reasons are from the canonical logs. A call that passes is signed (HMAC, key id, one-time nonce) and sent; an approved valve or grid call runs every check again.

#	Check	Refuses the call when	Reason logged
1	**schema**	the call does not match the contract	call does not match the contract: *field error*
2	**capability**	the Twin does not list the action	the Twin does not expose *action*
3	**allowlist**	the department may not use the action	department *name* may not call *action*
4	**incident open**	the incident is closed (recalls still pass)	incident *id* is closed (*outcome*)
5	**trust_ref**	no trusted, unexpired verdict covers its evidence	"no Guardian verdict referenced"
6	**evidence**	too few trusted sources for its impact level	"high-impact action needs 2 independent trusted sources, found 1 (wl-01-01)"
7	**human approval**	a valve or grid call has no signed approval yet	a high-impact action needs a person's approval...
8	**idempotency**	the same call was already issued	already issued for *key*
9	**cooling gap**	the actuator was driven too recently	*actuator* was last driven *n* s ago; the minimum gap...
10	**quarantine**	the device is quarantined or cannot be checked	"the Guardian has quarantined police-dispatch-05"
11	**budget**	6 actions per incident, or 20 per minute	incident *id* reached its limit of 6 actions

2.3 Tech stack and why

Everything runs offline on one laptop: the embedding model is in the Docker image, the screen loads nothing from the internet, and the scripted team answers when no model is reachable.

Table 2.2. The stack by job. Each logo belongs to its owner.

	Job and technology	Why
	Language and contracts Python 3.12, uv, Pydantic	One language for rules, agents and solvers; locked builds; every message a typed model that validates itself.
	Agents LangGraph, LangChain, Groq	Graphs that pause, resume and checkpoint, as the agents and human approval need; a scripted team answers when no model is reachable.
	Solvers OR-Tools, NumPy, NetworkX	Proven solvers with a certificate for answers that must be exact; the Kalman forecast; the city graph in memory.
	Storage Neo4j, Qdrant, SQLite	"Why was this unit sent?" in one graph query; hybrid search over 39 procedures; approvals and checkpoints. No decision waits on them.
	Interfaces MCP, FastAPI, MQTT, CloudEvents	MCP gives the Twin's tools schemas and discovery; REST is one question, one answer per verdict; SSE is one-way, replayable, library-free.
	Operations Docker, OpenTelemetry, three.js, pytest	One command starts the whole city; spans follow a reading through every step; the screen works offline; 1,114 tests.

2.4 Interfaces in/out

Direction	Message	Transport and contract
In, from the Twin	signed sensor readings; the city layout	MCP `list_readings(cursor)` every 200 ms and `get_topology()`; plain HTTP or MQTT as alternatives
In, from the Guardian	a signed verdict per reading; containment	contract 1.0: `POST /v1/verdicts/request` by sensor and sequence number, HMAC checked; `GET /v1/containment`
Out, to the Twin	signed calls (`dispatch_unit`, `set_pump_state`…)	dispatch, valve, grid and signal via the Guardian's audited `/v1/actuator-calls`; alerts, recalls and the pump to the Twin's MCP tools
Out, to the screen and partners	timeline events; the reasons for a decision; read-only tools	SSE `GET /events` (CloudEvents 1.0, 55 types in AsyncAPI); `/api/decisions/{id}/explanation`; MCP at `/mcp`

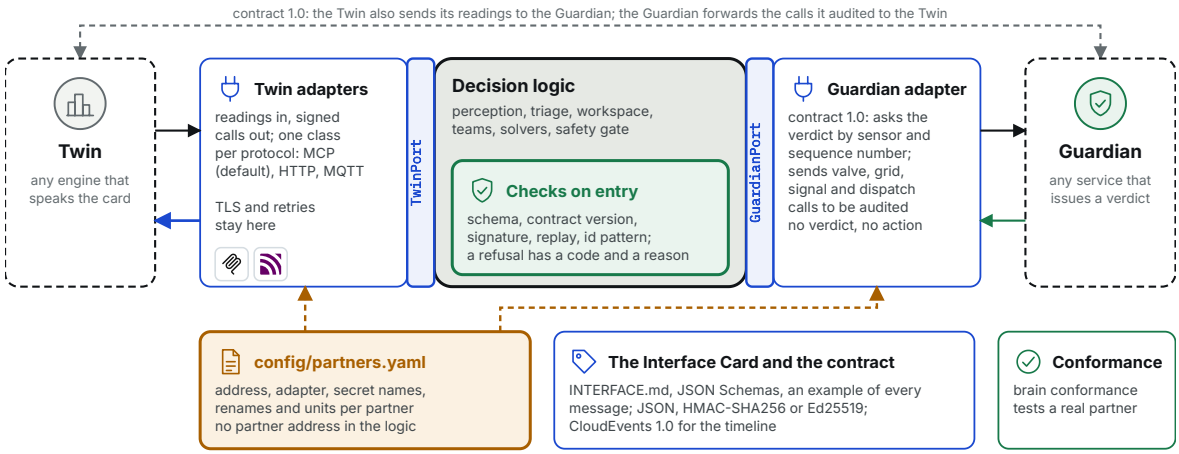


Figure 2.4. Where the logic ends and partners begin. Phase 2 reshuffles the triplets, so a new Twin or Guardian must be a new adapter class or an edit of `config/partners.yaml`, never a change to the decision logic, which reaches partners only through the two ports. The Twin's and the Guardian's sides of these links are in 1.4 and 3.4.

The jury looks for	Where the Brain does it, and the check that keeps it true
Logic separate from partner I/O	two ports in `ports.py`, one adapter class per protocol; `tests/test_layering.py` enforces it
A complete Interface Card	`INTERFACE.md`, an example of every message; `tests/test_contracts.py` validates each
Settings in configuration	`config/partners.yaml`, secrets from the environment; `brain onboard` maps a renamed Twin
Runs alone with mocks (R1)	`mocks/twin`, `mocks/guardian`; `brain conformance` runs 19 checks on a real partner

2.5 Security measures

The Brain treats every input as untrusted until it passes its checks, and every model answer as a proposal that code must verify. Valve and grid calls also need a person's signed approval. Threats are mapped to MITRE ATT&CK for ICS and, for the AI components, to MITRE ATLAS (`docs/SECURITY-MITRE.md`).

Listing 2.1. T13, a fake flood, from `logs/t13-20261108T153847Z.jsonl` (abridged). The gauge reads 1.80 m, the Guardian says untrusted, and the gate refuses both the warning and the valve.

```
15:38:51.522  brain.reading.received  wl-01-02  level 1.80 m
15:38:51.526  brain.verdict.received  verdict-000004  untrusted
15:38:51.526  brain.incident.opened   flood, zone-01, evidence untrusted
15:38:51.547  brain.action.blocked    broadcast_alert  trust_ref: no Guardian verdict referenced
15:38:51.548  brain.action.blocked    set_valve_state valve-01-01  trust_ref: no Guardian verdict referenced
15:38:51.975  brain.incident.closed   cleared: wl-01-01 still reads 0.30 m
```

Boundary	Control
Twin to Brain: readings	HMAC-SHA256 or Ed25519 over canonical JSON, key ids with expiry, a rising sequence number per sensor, schema and ranges; only allowlisted fields are kept, so demographic fields never reach a decision
Guardian to Brain: verdicts	schema check and the verdict's HMAC (contract 1.0); more independent trusted sources as impact rises
People and partners to the API	scoped bearer tokens off loopback (read, push, approve, admin), body and rate limits; a partner may push only `twin.*` or `guardian.*` events, and a forged `brain.*` event is refused
Brain to the language model	ids limited to safe characters before any prompt; read-only tools, with no actuating tool; models choose zone ids, not free text; code verifies the plan, then the gate checks it
Brain to Twin: calls	eleven gate checks; each call signed with its time and a one-time nonce; a valve or grid call needs a signed approval bound to its exact parameters and used once
Brain to its own disk	hash-chained log with a signed head; learning from trusted readings only; self-tuning limited to a listed set of parameters, each with a range

MITRE technique	Attack run against the code	Result
ICS T1692.002, forged reading	wrong key, no signature, a field edited after signing, a replayed sequence number, `alg: none`	refused every time
ICS T1692.001, T0849, forged call	an approval reused with changed parameters; the Twin pushing as the Guardian	refused
Enterprise T1190, T1078	no token, a forged token, a read token doing admin work, 28 path spellings with 7 methods	none got through
ATLAS AML.T0051, T0053	30 prompt-injection strings in sensor ids and values; a tool that tries to actuate	dropped before any prompt; no such tool exists

`brain redteam` runs 24 attacks; all were refused when re-run for this report. When the Guardian stops answering, the Brain acts on nothing it has not judged (T15), which is the Guardian's own fail-safe rule.

Reproduce: `uv run brain scenario t13 --inproc --speed 0`, `uv run brain redteam`, `uv run brain verify-log --scenarios`.

2.6 Known limitations

- **Joint runs, not yet complete.** On 8 November the Brain ran with the Twin team's Twin and the Guardian team's Guardian: three incidents end to end, a flood alert. Next: the same through Compose, with the agent teams.
- **The mock Guardian is scripted.** T13 and T16 show what the Brain does with a given verdict, not whether a real Guardian would issue it. Next: rerun both with `docker-compose.guardian.yml`.
- **Tuned on stand-ins.** Which sensors move together, and whether sensors in one zone are independent, waits for the real Twin's data. Next: record a normal day from it (`docs/TWIN-HANDOVER.md`).
- **TLS and the freshness limit are off by default.** Signatures still protect readings and calls. Next: set `guardian.tls` and `twin.max_age_s` on the live stack.
- **Guardian alerts are not read.** An unreliable actuator reaches the Brain through the containment list, which the gate obeys; a `compromised_twin` alert does not. Next: read `WS /v1/ws/verdicts`.

3. The Guardian

Written by the Guardian team · repository folder `triplet-03/guardian/` · contract 1.0 · key question: "Is this data telling the truth?"

3.1 Role and goals

The Guardian is the trust layer of the triplet. It judges what the city reports (the Twin's sensors and actuators) before the City Brain acts on it. The Brain cannot act safely on raw readings: data can be faked (a flood that is not there), replayed, frozen by a dead sensor, or drowned in a flood of emergency calls. The Guardian judges every reading and every actuator order, and contains what it finds.

Every reading meets seven checks (L1 to L6, with L3 split in two), and nine attack patterns are correlated across readings. The eleven Guardian scenarios of the Test Book (T1 to T11) pass, with their logs committed; the suite has 113 tests with 95% line coverage.

Threats covered

Threat	Detected by	Response
Sensor fault (stuck, dead, out of range, wrong unit)	L3a range and unit, L4 flatline, L6 heartbeat	*suspect* or *unknown*; isolate if persistent
Spoofed or false data (a fake flood, an impossible event)	L3b city model, L4 dynamics, L5 corroboration	*untrusted*, isolate, roll back
Replay and tampering	L1 HMAC and schema, L2 sequence numbers	*untrusted*; tampering or replay alert
Flooding (calls, messages)	L6 arrival rate, duplicate ids	throttle, corroborate, never drop
Actuator failure (ordered but not done)	effect verifier with an independent sensor	`actuator_failure`; mark it unreliable
Cyber-attacks across readings	threat correlator, nine patterns (3.2.3)	zone threat level, alert, quarantine
Unsafe or injected commands	call auditor: `trust_ref`, policies, rate limits	block; quarantine the caller

Design principle. Simple, explainable detectors that visibly catch the attack beat a black-box model. Every verdict names its reasons, so the Brain, the jury and the timeline can see why a reading was rejected. Optional machine learning can only lower trust, never raise it, and stays off the critical path.

3.2 Architecture

Four functional blocks share one state store, wrapped by adapters on the outside and an audit trail at the bottom.

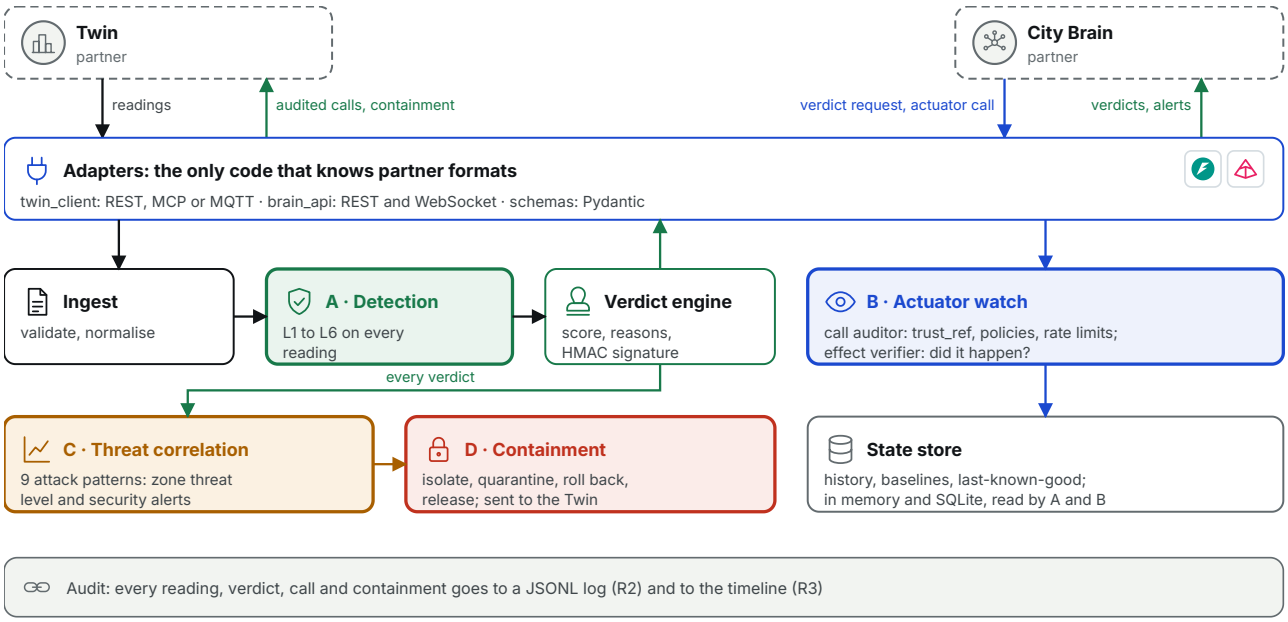


Figure 3.1. The Guardian's blocks. Readings flow all the time, from ingest through L1 to L6 to a verdict that threat correlation also reads. The Brain's verdict requests and actuator calls arrive on demand and pass the auditor, then the effect verifier.

3.2.1 Block A: the trust checks on every reading

Cheap, definitive checks run first; statistical and cross-sensor checks run last. Failing L1 or L2 makes a reading untrusted at once (a hard gate). L3 to L6 adjust a score that starts at 1.0.

Layer	Question	How it checks	Catches
L1 Authenticity	genuine and well formed?	Pydantic schema, `contract_version`, HMAC-SHA256 over a canonical body, constant-time compare	tampering, injection, malformed data
L2 Freshness	is it new?	`sequence_no` strictly rising per sensor, timestamp skew; an agreed `sequence_reset` flag after a Twin restart	replay attacks
L3a Range and unit	physically possible?	per-type ranges, internal consistency (load ≤ capacity), a unit heuristic (a value about 100× over range that fits after ÷100 is cm sent as m)	faulty values, unit mix-ups
L3b Semantic	possible here?	the city model in `city_model.yaml`: a fire in the open sea, a flood above a zone's maximum level, a sensor in a zone without that feature	impossible context, crude spoofing
L4 Dynamics	does the change make sense?	rate of change, flatline (N identical readings), EWMA and z-score, a per-hour baseline, slow same-sign drift	fake spikes, frozen sensors, drift
L5 Corroboration	does the city agree?	independent sources from `crosschecks.yaml` (rain, flow and the upstream level for water; neighbouring zones), a minimum source count	single-sensor and subtle spoofing
L6 Liveness and rate	alive, at a normal pace?	no reading within twice the expected interval is stale; more than 5 emergency calls per zone per minute is a flood; a duplicate `call_id`	dead devices, call floods

Scoring (`thresholds.yaml`): plausibility fail −0.5, semantic suspect −0.3, dynamics fail −0.3, corroboration fail −0.5, corroboration pass +0.2, liveness fail −0.3, pattern fail −0.2. A score of 0.8 or more is *trusted*, 0.4 to 0.8 *suspect*, below 0.4 *untrusted*; stale or missing data is *unknown*. In a zone whose threat level is raised, *trusted* needs 0.9 and two independent corroborating sources, and a reading with a big move needs corroboration to reach *trusted*.

Example (T1, from the committed log). The port sensor `wl-port-03` reports 1.80 m while the real level is 0.40 m and there is no rain. Within 10 s the verdict is *untrusted* with the reasons `rate_of_change`, `no_rainfall_corroboration`, `no_flow_corroboration` and `no_upstream_level_corroboration`; the sensor is isolated; `/v1/readings/wl-port-03/latest` serves the rolled-back 0.41 m; and the Brain mock ignores the reading and leaves the valve alone.

3.2.2 Block B: actuator watch, ordered against happened

The Brain sends its valve, grid, signal and dispatch calls through the Guardian (`POST /v1/actuator-calls`). A Twin reply of `status: ok` only means the call was accepted: a stuck valve or a lying controller can answer ok and do nothing.

Check	Detail
1. Authorisation	the call carries a `trust_ref` to a verdict that is trusted and not expired (30 s); no `trust_ref`, or a quarantined target, is blocked
2. Policy and safety invariants	from `policies.yaml`: no grid disconnect in the hospital zone, no repeated dispatch (same incident, station, unit, mode and zone), rate limits per caller and per actuator
3. Effect verification	an expectation with a deadline and a tolerance, watched by an independent sensor: valve closed, flow drops at least 80% within 30 s; grid disconnected, the zone's load near 0; signal green, traffic changes. No effect raises `actuator_failure` and marks the actuator unreliable, so further calls to it are refused
4. Audit	every call, decision and justification is logged and shown on the timeline

If the verifying sensor is itself not trusted, the result is *unknown*: the Guardian never accuses an actuator on bad evidence (T5).

3.2.3 Block C: threat correlation

Checks on single readings cannot see an attack spread over many. The correlator groups verdicts and calls by zone, device, source and time window against the catalogue in `attack_patterns.yaml`. It raises a zone's threat level from *normal* to *elevated* to *under_attack*, which tightens the trust bar, and emits a security alert with its type, evidence and suggested response. Levels decay with hysteresis after 120 s of quiet.

Pattern	Signal	Response
`replay`	a repeated `sequence_no` with a valid HMAC	untrusted; flag the source
`spoofing`	failed corroboration or impossible dynamics	isolate, roll back
`coordinated_spoofing`	several sensors in one zone fail within a window	zone *under_attack*, stricter trust
`flood_dos`	a message-rate spike, an emergency-call flood	throttle; mark for corroboration, never drop
`tampering`	HMAC failures, schema violations	reject; possible key compromise alert
`command_injection`	calls without `trust_ref`, bursts	block; quarantine the caller after 3
`slow_drift`	a small same-sign deviation over a long window	alert; freeze the baseline
`compromised_twin`	signatures valid, but corroboration collapses city-wide	alert advising safe mode
`attack_during_incident`	a real corroborated event and a fake alarm elsewhere	keep acting on trusted data only

3.2.4 Block D: containment

Action	Effect	Guardrail
`isolate_sensor`	its readings can no longer be used by the Brain	a cap per zone; evidence logged
`quarantine_device`	a sensor, actuator or caller is blocked	reversible; a reason is required
`rollback`	serves the last known good value, marked `rolled_back: true`	visible in the log and the API
`clear_quarantine`	release after clean readings	hysteresis, a growing cool-down, a manual release endpoint

Containment goes to the Twin as calls with `requesting_layer: guardian`; the Guardian authorises its own containment. Caps and release paths stop an attacker from turning quarantine against healthy sensors.

3.3 Tech stack and why

Table 3.1. The Guardian's stack. Each logo belongs to its owner.

	Concern and choice	Why
	Language Python 3.11+, asyncio	fast to write detection logic; a non-blocking pipeline
	Brain-facing API FastAPI (REST and WebSocket), OpenAPI at `/docs`	standard, self-documenting, validated input
	Twin transport REST push or pull by default; MCP and MQTT adapters	the Twin may expose MCP-style tools or a stream; switched by configuration
	Validation Pydantic v2	strict schemas and precise error messages
	Crypto standard-library `hmac` and `hashlib` (SHA-256)	no custom crypto; constant-time compare
	State in-memory ring buffers and SQLite	no external infrastructure; survives a restart
	Statistics NumPy; scikit-learn Isolation Forest, off by default	light and explainable; the model can only lower trust
	Config, logs 7 YAML files and `.env`; structlog JSONL, WebSocket, an HTML screen	nothing partner-specific in code; machine-checkable logs (R2), a live view (R3)
	Quality, packaging pytest, hypothesis, ruff, mypy; Docker Compose with both mocks	unit, integration, fuzz and scenario tests; one command, nothing installed on stage, runs offline

Left out on purpose: Kafka, Kubernetes, deep learning and external databases, which add demo risk without improving a trust decision.

3.4 Interfaces in/out

Direction	Message and transport	Purpose
Twin → Guardian	the signed sensor envelope: `POST /v1/readings` (push), or pull over HTTP, MCP `get_readings` or MQTT `city/readings/#`	the evidence to judge
Brain → Guardian	`POST /v1/verdicts/request`	the check before acting
Guardian → Brain	the signed verdict in reply; verdicts and alerts also on `WS /v1/ws/verdicts`	safe decisions
Brain → Guardian → Twin	`POST /v1/actuator-calls` with a `trust_ref`, forwarded to `POST {twin}/tools/{action}`	the audited actuator gateway
Guardian → Twin	containment calls (isolate, quarantine, clear)	the city really changes
Guardian → everyone	`WS /v1/ws/timeline`, the `/timeline` screen, JSONL logs	cause and effect (R2, R3)

Listing 3.1. A verdict, from the Guardian to the Brain (abridged). Every message has an example in the Interface Card, `guardian/INTERFACE.md`.

```
{ "verdict_id": "verdict-88213",
  "reading_ref": {"sensor_id": "wl-port-03", "sequence_no": 4821},
  "status": "untrusted", "trust_score": 0.2,
  "reasons": ["rate_of_change", "no_rainfall_corroboration", "no_flow_corroboration"],
  "layer_results": [ {"layer": "L1_authenticity", "passed": true, ...}, ... ],
  "action_taken": "isolate_sensor", "threat_level": "normal",
  "ts": "...", "expires_at": "...", "signature": "<HMAC-SHA256>" }
```

Other endpoints: `/health`, `/v1/readings/{id}/latest` (the rolled-back value when contained), `/v1/sensors/{id}/status`, `/v1/threat`, `/v1/alerts`, `/v1/containment` (with `/clear` and `/quarantine`), `/v1/verify/signature`.

Repointing to another partner (Phase 2) means changing `twin.base_url`, `twin.transport` (rest, mcp or mqtt) and `twin.mode` (push or pull) in `config/guardian.yaml` or the environment, and the shared keys in `.env`. In the triplet's Compose file the Guardian also accepts the shared names `GUARDIAN_URL` (port 8003), `TWIN_URL` and `TWIN_HMAC_SECRET`. No code changes.

Input validation and errors. Every endpoint answers malformed input with a 4xx and one body shape, `{"error", "message", "field"}`; the codes include `schema_invalid`, `contract_version_mismatch`, `hmac_invalid`, `auth_required`, `trust_ref_missing` and `verdict_not_found`. A well-formed reading with a bad HMAC is not an error but an untrusted verdict. A bad configuration file stops start-up and names the file and the field.

3.5 Security measures

- **Authenticated API.** One API key per partner (`X-API-Key`; `?api_key=` for browser WebSockets, closed with 4401 on a bad key). The key decides the caller identity used in audit and quarantine.
- **Signed data both ways.** Sensor envelopes are checked with the Twin's HMAC key, and verdicts are signed with the Guardian's key, so the Brain can detect a forged verdict. Both use the same canonical form, and an endpoint checks it.
- **Replay protection.** Strict sequence numbers per sensor and a timestamp skew limit; an explicit, signed `sequence_reset` for a legitimate restart.
- **A gate on actions.** Every actuator call needs a fresh trusted verdict and passes the safety invariants and rate limits; a caller that injects commands is quarantined.
- **Fail-safe.** If the Guardian is unreachable or a verdict is missing, expired or not trusted, the Brain treats the data as unknown and does not act. An internal error gives *unknown*, never *trusted*. A silent Twin turns every sensor unknown after twice its interval (T7).
- **Proportionate response.** Isolation caps per zone, hysteresis, a growing cool-down and a manual release path, so the defence cannot be turned against healthy sensors.
- **Secrets out of code.** Keys live only in `.env` (git-ignored), and the service refuses to start without them.

Handling of false alarms

Risk	How the Guardian avoids it
A real event looks abrupt (a burst pipe, a real flood)	dynamics alone only gives *suspect*; corroboration decides. The real flood in T11 stays trusted
An incomplete city map	semantic failures are *suspect* unless physically impossible (a fire in the open sea)
A real disaster brings many calls	call floods are throttled and marked for corroboration, never dropped
Its own order changes a sensor (valve closed, flow drops)	a change explained by a call the Guardian forwarded is not flagged during the expectation window
A bad verification sensor; a Twin restart	an untrusted verify sensor gives *unknown*; a signed `sequence_reset` is accepted instead of raising replay
Flapping isolation	release after clean readings, hysteresis, a growing cool-down, caps per zone

3.6 Known limitations

- **A compromised Twin.** If the Twin itself lies, HMAC does not help. Corroboration is the defence and weakens as more sensors are spoofed together; a city-wide collapse raises `compromised_twin` but cannot say which value is true.
- **HMAC proves origin, not truth.** A compromised sensor signs lies perfectly; that is why L3 to L6 exist.
- **Two sources for trusted.** A notable reading needs two independent sources; a sensor with no corroboration path can only be *suspect*.
- **Thresholds.** They trade false alarms against missed attacks; the defaults suit the mock city and need tuning on the real Twin.
- **Baselines learn online** from trusted readings only (at least 200 samples per hour bucket); an attack present from the very start is caught only by the other layers. A recorded normal day from the Twin would help.
- **A stolen Brain key** could be used to get a caller quarantined.
- **One process.** State is in memory and SQLite; several replicas would need shared state. Effect windows are fixed per action.

4. Test Book

All three teams · one numbering: T1 to T11 the Guardian team's, T12 to T16 the City Brain team's, T17 to T20 the Twin team's · every field (description, trigger, expected output per layer, how to verify) in `TESTBOOK.md`

Taken together the scenarios test all three layers and every interaction between them. Each starts from one trigger and leaves one log, so the jury can check a run after the event.

Interaction	What it shows	Scenarios
Twin → Brain	the city runs normally and the Brain reacts correctly to real events	T12, T14, T17 to T20
Twin → Guardian	the Guardian recognises healthy and unhealthy data	T1 to T4, T7 to T9, T13
Brain → Guardian	trust verdicts change decisions, and decisions stay safe	T5, T6, T10, T13, T14, T16
All three	the city works as one system, including when something goes wrong	T11, T15

Team	Trigger	Log
Guardian	`make scenario T=T1` in `guardian/`	`logs/t1-guardian-*.log`
City Brain	`brain/scenarios/triggers/t12.sh`, or a number key on the screen	`logs/t12-*.jsonl`, hash-chained
Twin	`docker compose run --rm scenario t1`, or a button in the viewer	`logs/t1-*.log`, ending in PASS or FAIL

ID	Team	Scenario	Expected, observable
T1	**Guardian**	Fake flood at the port (1.80 m vs 0.40 m, no rain)	untrusted within 10 s, sensor isolated, rollback serves 0.40 m; the Brain does not move the valve
T2	**Guardian**	Stuck or dead sensor	flatline gives suspect; silence gives stale, then unknown
T3	**Guardian**	Replayed readings	a repeated sequence_no with a valid HMAC is untrusted (replay); a signed sequence_reset is accepted
T4	**Guardian**	Emergency-call flood	over 5 calls per zone per minute: call_flood, suspect, throttled, none dropped
T5	**Guardian**	Valve closed, no effect	actuator_failure alert; with an untrusted flow sensor the result is unknown instead
T6	**Guardian**	The Brain uses a quarantined device	call blocked, reason device_quarantined
T7	**Guardian**	Twin goes silent	every sensor unknown, /health degraded
T8	**Guardian**	Fire in the open sea	untrusted (impossible_location), dispatch blocked
T9	**Guardian**	Coordinated spoofing in a zone	zone under_attack: trusted needs 0.9 and 2 sources; back to normal with hysteresis
T10	**Guardian**	Command injection without trust_ref	blocked; the caller quarantined after 3; command_injection alert
T11	**Guardian**	Real flood and an attack elsewhere	the corroborated flood trusted, the valve call forwarded and verified; the fake alarm untrusted

ID	Team	Scenario	Expected, observable
T12	**City Brain**	A calm day	no incident and no actuator call: the Brain does not overreact
T13	**City Brain**	A fake flood	the gauge is untrusted; the gate refuses the warning and the valve; nothing reaches the city
T14	**City Brain**	A crime and a pursuit	reflex dispatch with a trust_ref, a solver-checked perimeter, every unit recalled at the arrest
T15	**City Brain**	A flood, an attack and a layer down	warning sent, valve held, no police for the forged report; once the Guardian is silent, no action on unjudged readings
T16	**City Brain**	A quarantined police station	every call to the quarantined station refused (quarantine); the pursuit unit leaves another station
T17	**Twin**	A car hits a walker (Part 1: T1)	a witness calls, the signed emergency_call is trusted, the Brain dispatches an ambulance and police with a trust_ref, the street closes and reopens; 10 of 10 expected events, PASS at 245 s
T18	**Twin**	Part 1: T2, at junction 2	10 of 10 expected events of the three layers, PASS at 287 s
T19	**Twin**	Part 1: T3, at a building in block A	9 of 9 expected events, PASS at 146 s
T20	**Twin**	A street floods at its low point (Part 1: T4)	12 of 12 expected events, PASS at 340 s

The Guardian’s scenarios run against its own Twin and Brain mocks, the Brain’s against its mocks of the Twin and the Guardian, and the Twin’s against its mocks of the other two. The Twin team numbers its scenarios T1 to T4 in Part 1; T18 and T19 are described here only as far as Part 1 describes them.